DevTrack: Design Notes

A Django backend API for tracking engineering issues. Architecture overview.

Request path: one layer, one job

Client (Postman)
Sends JSON over HTTP, e.g. POST /api/issues/

↓

urls.py (project + issues)
/api/ goes to issues/urls.py, which routes reporters/ and issues/

↓

views.py
Reads the request, checks types, picks the class, builds the response and status code

↓

models.py
OOP classes and business rules: validate(), to_dict(), describe()

↓

storage.py
load(), save() (atomic), next_id()

↓

reporters.json / issues.json
The data: plain JSON lists in the project root

What each layer does, and never does

views.py

Does
HTTP in and out, type checks, status codes, picks the Issue class.
Never
Inspect names or emails itself. Open files.

models.py

Does
Business rules, serialization (to_dict), describe(). Plain Python.
Never
Know about HTTP or files. No Django imports.

storage.py

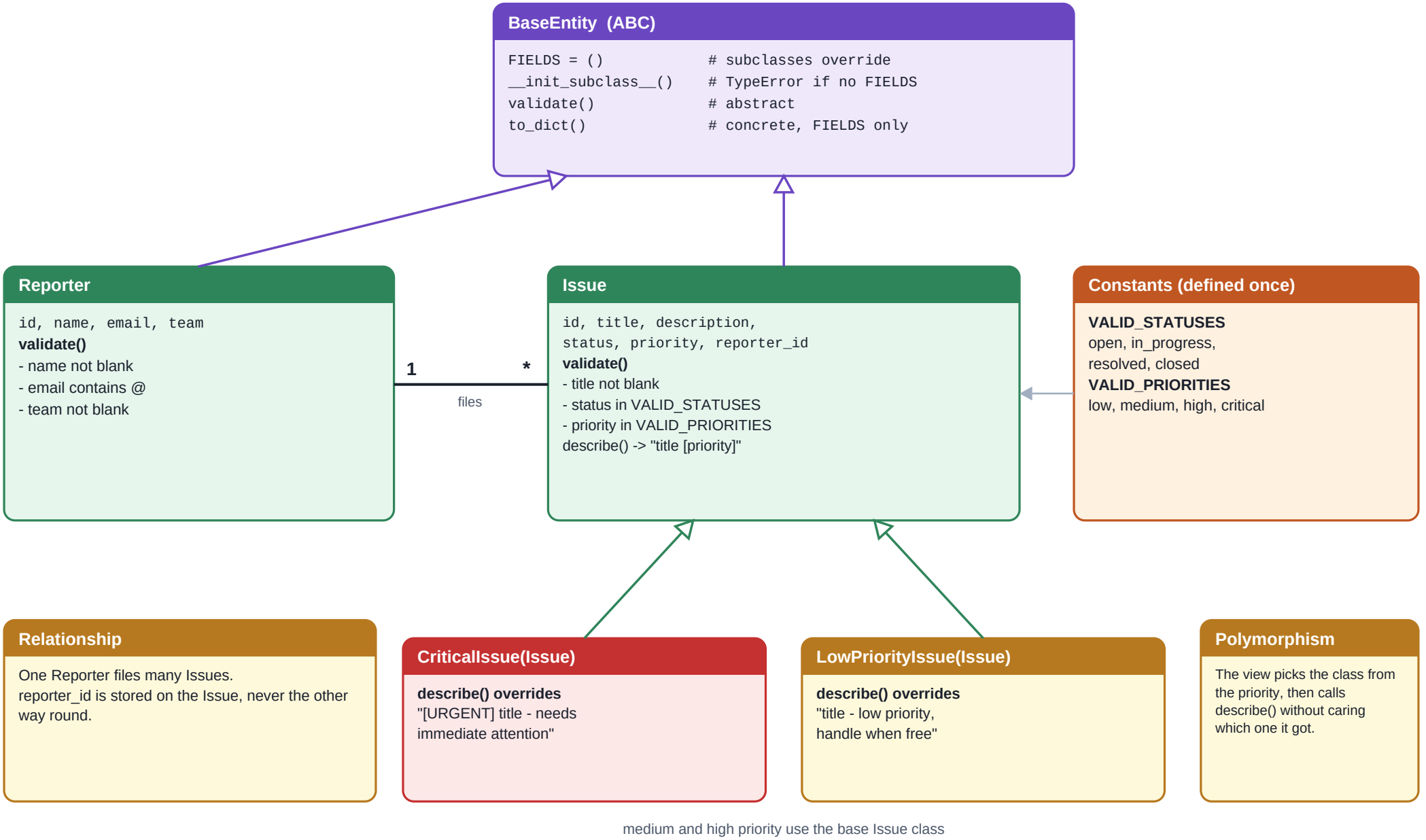
Does
Read and write JSON, hand out the next ID.
Never
Validate data or know about HTTP.

Why this split pays off

Swap JSON for a database: only storage.py changes.
Add a new priority class: one subclass plus one entry in PRIORITY_CLASSES.
Change a business rule: edit validate(), never the view.

OOP Design

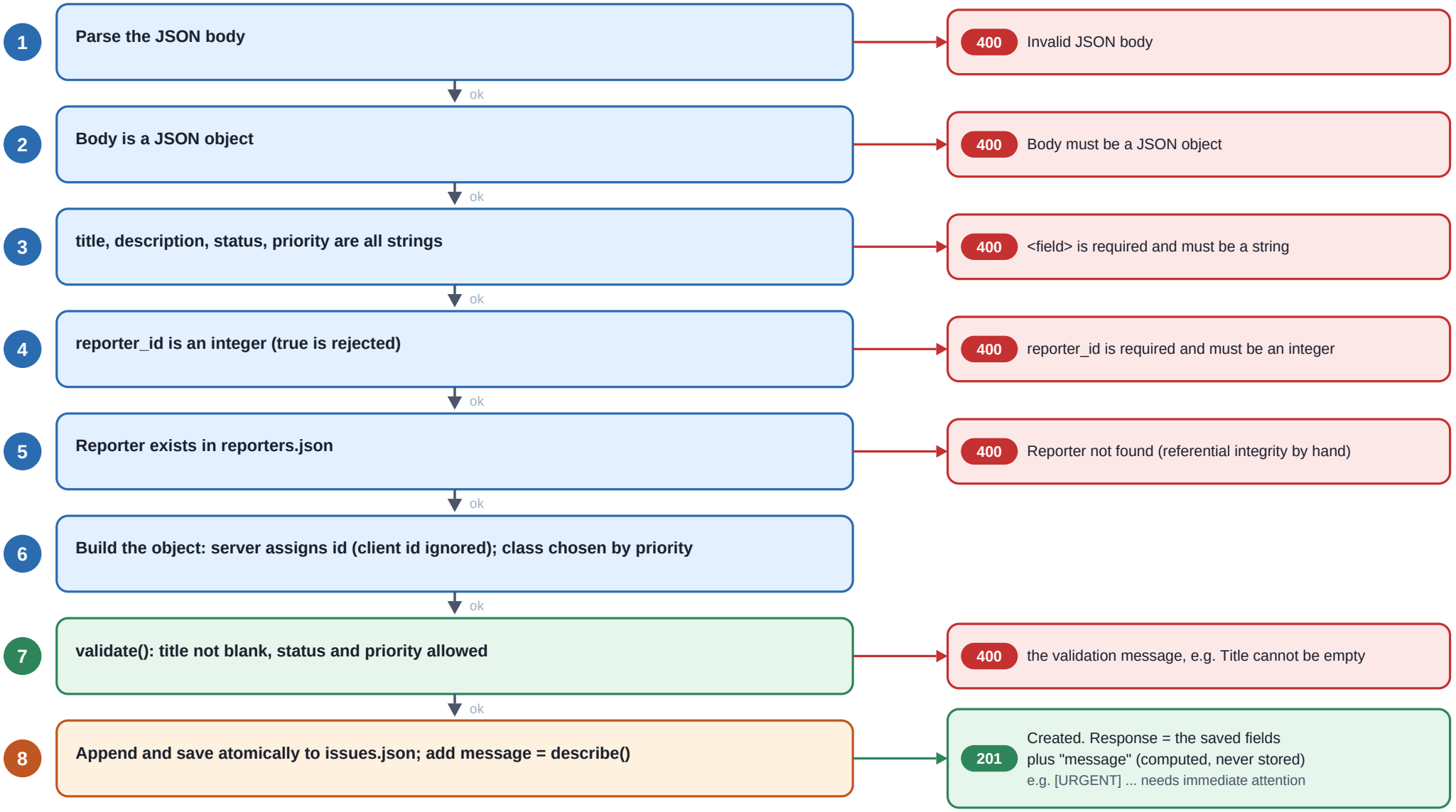
Two entities, one abstract base, two priority subclasses. All in issues/models.py.



medium and high priority use the base Issue class

POST /api/issues/

What happens to a request, step by step. Any failing step stops the request.

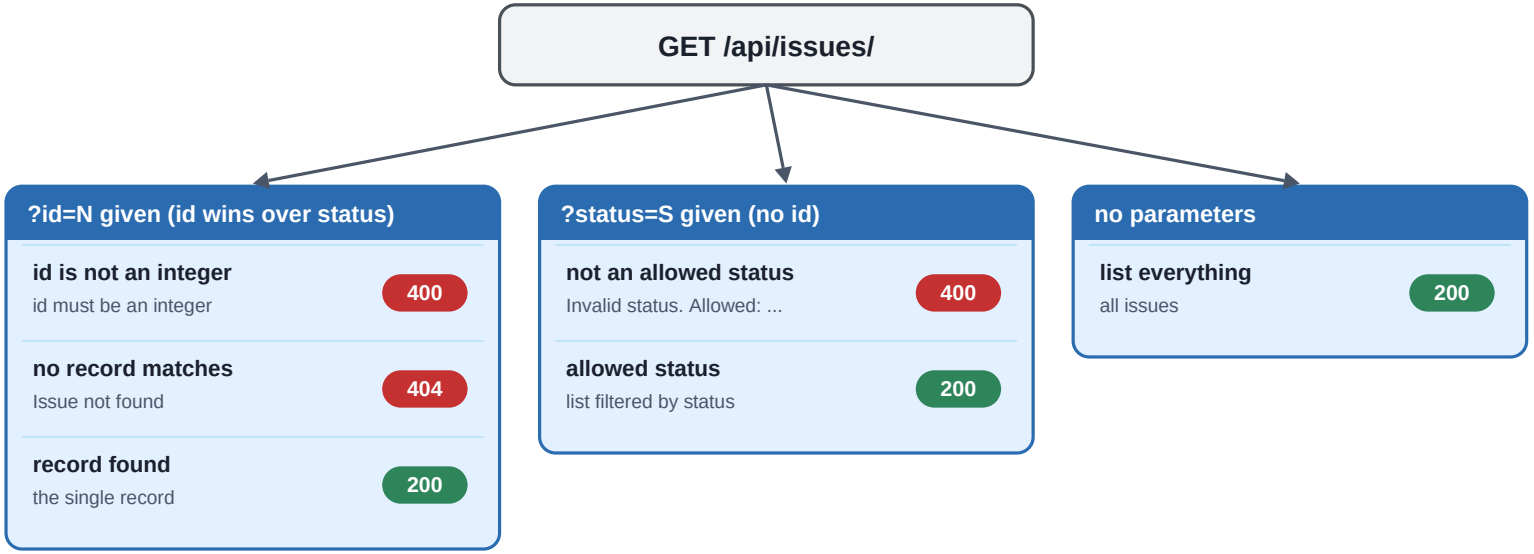


views.py models.py storage.py

Two layers of validation: view checks types (steps 2-5), model checks business rules (step 7).

GET Endpoints and Status Codes

How the query parameters are interpreted. Reporters follow the same pattern (id only).



Why an unknown reporter_id gives 400, not 404

404: what the URL points at is missing

`GET /api/issues/?id=99`
The URL asks for issue 99. No such issue exists.
Response: 404 Issue not found

400: URL fine, request body is wrong

`POST /api/issues/`
`{ ..., "reporter_id": 99 }`
The endpoint exists. The body points at a reporter that does not.
Response: 400 Reporter not found
422 is another common choice; the brief only specifies 400.

Status codes

200 OK
Successful GET.

201 Created
A reporter or issue was saved.

400 Bad request
Invalid JSON, wrong type, failed validation, non-integer id, bad status filter, unknown reporter_id.

404 Not found
?id= matched no record.

405 Method not allowed
Wrong HTTP method on a URL. Empty body, no JSON.

Every API error body has the shape `{"error": "<message>"}` (except 405)

Design Decisions (1 of 2)

Server-generated IDs and atomic writes.

Server-generated IDs: any id sent by the client is ignored

Client sends

```
{ "id": 50,
  "title": "Login bug", ... }
```

Server (create_issue)

Never reads data["id"].

```
id = max(existing ids) + 1
```

Stored and returned

```
{ "id": 3, "title": ... }
```

The response always shows the real id.

Why not trust client ids: Collisions

Two clients send the same id and the data becomes ambiguous.

Why not trust client ids: Overwrite

A client could guess an id and replace someone else's record.

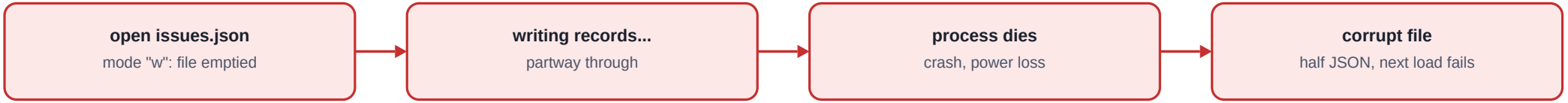
Why not trust client ids: Blind

A client cannot know the current maximum while others are writing.

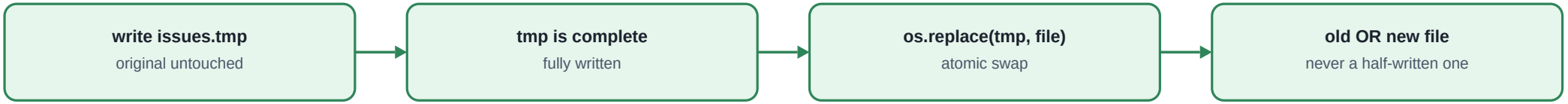
Also: max + 1, not len + 1, so ids stay unique even if records are ever removed. The brief's sample sends "id": 1, so this is a deliberate deviation (documented in the README).

Atomic writes: temp file, then os.replace

Unsafe: write straight to issues.json



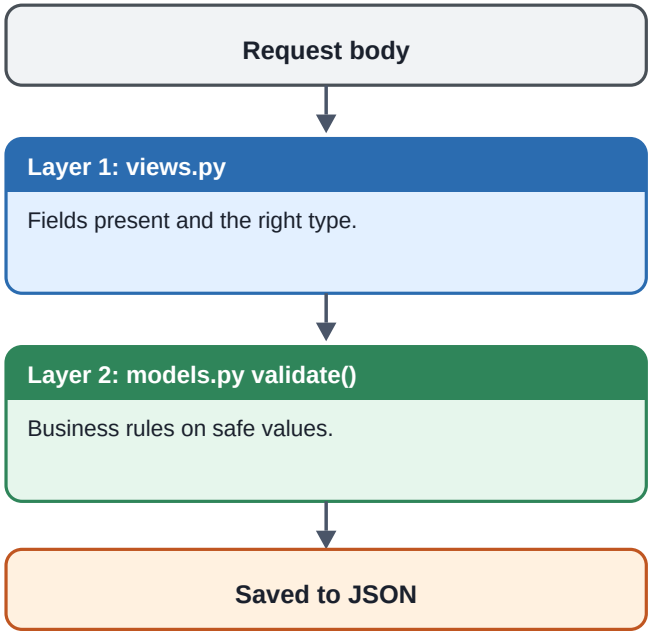
Safe: what storage.save() does



Design Decisions (2 of 2)

Two-layer validation, the FIELDS allowlist, and the known limitation.

Two layers of validation



catches: missing, null, numbers, bad JSON

catches: blank title, bad status, email without @

Layer 1 means validate() never crashes on None or a number.

to_dict(): FIELDS allowlist, not __dict__

Unsafe: return self.__dict__

```
attributes: id, title, ..., _cache
response : id, title, ..., _cache
```

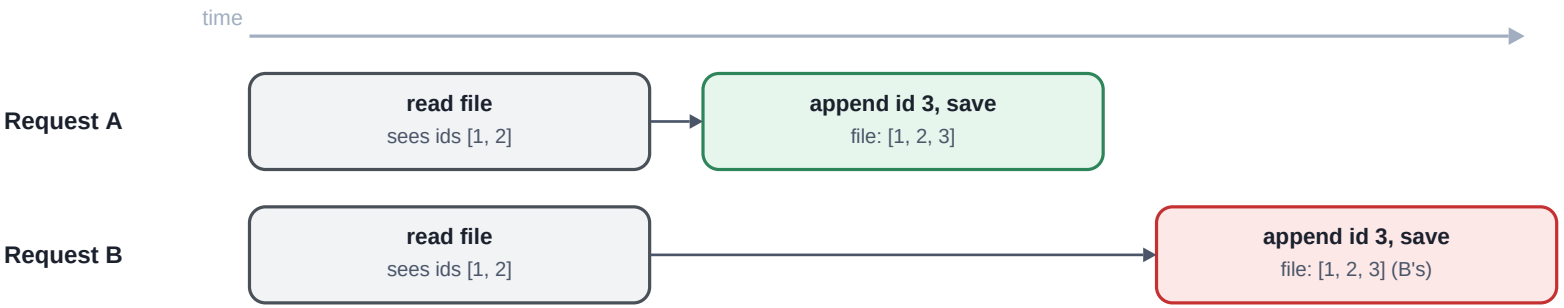
Every new attribute leaks into the API automatically.

Safe: loop over FIELDS

```
FIELDS: id, title, ..., reporter_id
response: id, title, ..., reporter_id
```

New attributes stay hidden until opted in. A missing FIELDS raises TypeError at import.

Known limitation: race condition (JSON files, no locking)



Result

A's issue is lost.
Both requests were told id 3.

Real fix

A database: transactions and unique ids. Acceptable for this project, and the main reason real systems do not use JSON files.